

LAB 1

DevOps Foundations & Continuous Integration

Task 5 - Set Up Version Control Using Git

Student: **Abhay rana**

Objective

Initialize a Git repository, stage and commit changes, create and work with a branch, merge the branch into main, configure a GitHub remote, push the repository to GitHub, and verify the final repository status.

Project: ~/lab1-devops **Remote:** GitHub lab1-devops-version-control

Evidence sequence

The screenshots are arranged chronologically so each image directly supports the corresponding step of the required Git version-control workflow.

1. Repository location and initial project file

The working directory is `~/lab1-devops`. The **README.md** file is present, establishing the project before Git initialization.

```
abhay_rana@LAPTOP-45509TK7:~/lab1-devops$ echo "Student Name: Abhay"
Student Name: Abhay
abhay_rana@LAPTOP-45509TK7:~/lab1-devops$ pwd
/home/abhay_rana/lab1-devops
abhay_rana@LAPTOP-45509TK7:~/lab1-devops$ ls
README.md
abhay_rana@LAPTOP-45509TK7:~/lab1-devops$ |
```

2. Initialize the Git repository

The command **git init** initializes the local Git repository. **git status** confirms the repository is on the initial **master** branch with README.md untracked.

```
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /home/kunal_prajapati/lab1-devops/.git/
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
      README.md

nothing added to commit but untracked files present (use "git add" to track)
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ |
```

Figure 2. 2. Initialize the Git repository.

3. Stage files for commit

The command **git add README.md** stages the project file. The status output shows README.md under **Changes to be committed**.

```
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ git add README.md
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   README.md

kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ |
```

Figure 3. 3. Stage files for commit.

4. Create the initial commit

The command **git commit -m "Initial project setup"** creates the first commit. The Git log confirms commit **ee06e58**.

```
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ git commit -m "Initial project setup"
[master (root-commit) ee06e58] Initial project setup
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ git log --oneline
ee06e58 (HEAD -> master) Initial project setup
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ |
```

Figure 4. 4. Create the initial commit.

5. Create a feature branch and commit changes

The **feature-notes** branch is created and a version-control note is committed. The log shows both the feature branch commit and the initial main/master history.

```
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ git branch -M main
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ git switch -c feature-notes
Switched to a new branch 'feature-notes'
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ echo "Git version control experiment completed." >> README.md
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ git add README.md
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ git commit -m "Add version control notes"
[feature-notes f7e9d4f] Add version control notes
1 file changed, 1 insertion(+)
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ git log --oneline --all --decorate
f7e9d4f (HEAD -> feature-notes) Add version control notes
ee06e58 (main) Initial project setup
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ |
```

Figure 5. 5. Create a feature branch and commit changes.

6. Merge the feature branch into main

The terminal shows **git switch main** followed by **git merge feature-notes**. The merge completes successfully and the graph shows **HEAD -> main** at the feature commit.

```
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ git switch main
Switched to branch 'main'
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ git merge feature-notes
Updating ee06e58..f7e9d4f
Fast-forward
 README.md | 1 +
 1 file changed, 1 insertion(+)
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ git log --oneline --all --decorate --graph
* f7e9d4f (HEAD -> main, feature-notes) Add version control notes
* ee06e58 Initial project setup
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ |
```

Figure 6. 6. Merge the feature branch into main.

7. Configure the GitHub remote

The GitHub repository is configured as the **origin** remote. **git remote -v** verifies both fetch and push URLs.

```
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ git remote add origin https://github.com/kunalprajapati14042005/lab1-devops-version-control.git
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ git remote -v
origin  https://github.com/kunalprajapati14042005/lab1-devops-version-control.git (fetch)
origin  https://github.com/kunalprajapati14042005/lab1-devops-version-control.git (push)
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$
```

Figure 7. 7. Configure the GitHub remote.

8. Push the repository to GitHub

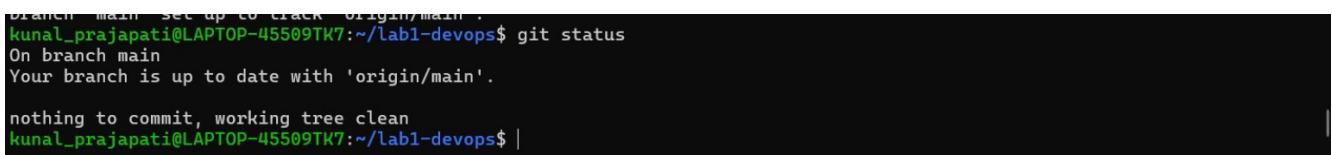
The `git push -u origin main` command successfully pushes the local main branch to GitHub and sets `origin/main` as the upstream branch.

```
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ git push -u origin main
Username for 'https://github.com': kunalprajapati14042005
Password for 'https://kunalprajapati14042005@github.com':
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (6/6), 549 bytes | 549.00 KiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/kunalprajapati14042005/lab1-devops-version-control.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ |
```

Figure 8. 8. Push the repository to GitHub.

9. Final repository status

The final **git status** confirms the local main branch is up to date with **origin/main** and the working tree is clean.

A terminal window with a dark background. The prompt is 'kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops\$'. The command 'git status' has been entered. The output is: 'On branch main', 'Your branch is up to date with \'origin/main\'', and 'nothing to commit, working tree clean'. The prompt is now 'kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops\$ |'.

```
branch main set up to track 'origin/main'.
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ |
```

Figure 9. 9. Final repository status.

Conclusion and Requirement Mapping

Task 5 requirement: initialize a repository, stage and commit changes, create branches, merge branches, and push the repository to GitHub.

Required activity	Evidence
Initialize repository	Figure 2 - git init and git status
Stage changes	Figure 3 - git add and staged status
Commit changes	Figure 4 - initial commit and log
Create branch	Figure 5 - feature-notes branch
Merge branch	Figure 6 - switch main and merge feature-notes
Configure GitHub remote	Figure 7 - git remote -v
Push to GitHub	Figure 8 - successful git push
Verify final state	Figure 9 - clean, up-to-date git status

Conclusion

Task 5 was completed successfully. The evidence demonstrates the complete local Git workflow from repository initialization through staging, commits, branching, merging, remote configuration, GitHub push, and final clean-status verification.